Markov Decision Processes and Policy / Value Iteration

- **Markov Decision Processes** model environments where *future states* are independent of *past states* given the *present state*
  - $P^\pi(s'|s) = \sum_{a\in A} \pi(a|s)P(s'|s,a)$
  - $R^\pi(s) = \sum_{a\in A} \pi(a|s)R(s,a)$
- **Value Function** is the expected return starting from state s and following policy π while the **Action-Value Function** $q^\pi(s,a)$ is the expected return starting from state s and taking action a, and then following policy π
  - $v^\pi(s) = \sum_{a\in A} \pi(a|s)q^\pi(s,a)$
  - $q^\pi(s,a) = R_s^a + \gamma \sum_{s'\in S} P(s'|s,a)v^\pi(s')$
  - Bellman Relationships:
    - $v^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v^\pi(s_{t+1})|s_t = s] = \sum_{a\in A} \pi(a|s)(R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a)v^\pi(s'))$
      - Intuition: Marginalize over all possible actions (based on distribution from policy) - getting the expected return based on the immediate reward as well as the discounted future reward (which we must base on all possible next states given current state and the marginalized action)
    - $q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q^\pi(s_{t+1}, A_{t+1})|s_t = s, A_t = a] = R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a) \sum_{a'\in A} \pi(a'|s')q^\pi(s',a')$
      - Intuition: The immediate action is given (as a parameter), so we just need to marginalize the *next state* given the current state and chosen action; from there, we average over all action-value functions as well
- Given a policy and the full environment dynamics, the value function associated with that policy can be computed via dynamic programming - **synchronous backup**
  - $v_{t+1}(s) = \sum_{a\in A} \pi(a|s)(R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a)v_t(s'))$
- To compute the **optimal policy**, we can maximize over the optimal state-action value function
  - **Policy Iteration**: Iterate between evaluating the policy π and improving it greedily with respect to v^π
    - Evaluation: Use synchronous backup to find v^π
    - Improvement:
      - Compute $q^{\pi_i}(s,a) = R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a)v^{\pi_i}(s')$
        - This gives a *q-table*, where there is an *expected reward* for each action in each state
      - Compute $\pi_{i+1}(s) = \text{argmax}_a\, q^{\pi_i}(s,a)$
    - Repeat until convergence (policy does not change) - this means $q^\pi(s,\pi(s)) = v^\pi(s)$
  - **Value Iteration**: Iterate directly to find the optimal v^π, and then extract the policy once afterwards
    - Repeat:
      - $q_{k+1}(s,a) = R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a)v_k(s')$
      - $v_{k+1}(s) = \max_a q_{k+1}(s,a)$
    - After convergence:
      - $\pi(s) = \text{argmax}_a[R(s,a) + \gamma \sum_{s'\in S} P(s'|s,a)v_{k+1}(s')]$
- The aforementioned algorithms *only work if the model (environment) dynamics are known*

Model-Free Prediction and Control

- **Monte Carlo Policy Evaluation**: Sample trajectories, compute the returns, and average them to estimate $v^\pi(s)$
  - This works well, even in known environments, if transition probabilities are too complex to compute
  - Algorithm:
    - Loop:
      - Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T_1}, A_{T_1}, R_T$
      - Loop for each step of episode backwards: $t = T-1, T-2, \ldots 0$:
        - $G \leftarrow \gamma G + R_{t+1}$
        - Append G to $Returns(S_t)$
        - $V(S_t) \leftarrow \text{average}(Returns(S_t))$
  - Can use an incremental mean: $\mu_t = \frac{1}{t}\sum_{j=1}^t x_j = \mu_{t-1} + \frac{1}{t}(x_t - \mu_{t-1})$
- **Temporal Difference Learning**: Uses *boostrapping*, which involves updating an estimate using *other estimates* (like in dynamic programming) rather than waiting for the whole episode to complete. This is advantageous in that it can learn from *incomplete* sequences and *non-terminating* environments
  - **TD(0)**: $v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t))$
    - View $[R_{t+1} + \gamma v(S_{t+1})] - v(S_t)$ as the *error* of the existing estimate, which acts as a correction
  - **n-step**: $G_t^n = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1}R_{t+n} + \gamma^n v(S_{t+n})$
    - $v(S_t) \leftarrow v(S_t) + \alpha(G_t^n - v(S_t))$
- **Generalized Policy Iteration**:
  - **Monte Carlo with Exploring Starts**:
    - Loop Forever:
      - Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
      - Generate an episode from S_0, A_0 following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_t$
      - Loop for each step of episode backwards: $t = T-1, T-2, \ldots, 0$:
        - $G \leftarrow \gamma G + R_{t+1}$
        - Append G to $Returns(S_t, A_t)$
        - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
        - $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$
  - **Monte Carlo with Epsilon-Greedy Exploration**:
    - ϵ-Greedy Exploration puts probability $1-\epsilon$ to choose the greedy (optimal) action and probability ϵ to choose a random action - the goal is to ensure there is sufficient exploration being done rather than exploiting a (potentially bad) policy
    - Algorithm:
      - Loop:
        - Sample $(S_1, A_1, R_2, \ldots, S_T) \sim \pi_k$
        - For each S_t and A_t:
          - $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$
          - $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}(G_t - Q(S_t, A_t))$
            - This is the incremental mean
        - $k \leftarrow k+1,\ \epsilon \leftarrow 1/k$
          - Anneal ϵ
        - $\pi_k = \epsilon - \text{greedy}(Q)$
  - **On-Policy TD Control (SARSA)**: Estimates Q using TD
    - Repeat for each episode:
      - Initialize S
      - Choose A from S using ϵ-greedy policy
      - Repeat (for each step of episode):
        - Take action A, observe R, S'
        - Choose A' from S' using ϵ-greedy
        - $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$
          - This can be generalized to n-step as well: $q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1}R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$
        - $S \leftarrow S'; A \leftarrow A'$
  - **Off-Policy TD Control (Q-Learning)**: Use *two different policies* - one being learned about and becomes optimal policy, and another which is more exploratory (used to generate trajectories); doing this allows for reuse of old experiences
    - Repeat for each episode:
      - Initialize S
      - Repeat (for each step of episode):
        - Choose A from S using ϵ-greedy derived from Q
          - This is the **behavior policy**, which is epsilon-greedy on the target Q
        - Take action A, observe R, S'
        - $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
          - This is the **target policy**, which is fully greedy on Q
          - The learning 'acts' as if it took the target policy when updating, but the steps actually taken in the environment solely use the behavior policy
        - $S \leftarrow S'$
- **Importance Sampling**: Rather than sampling from a given distribution, it may be easier to sample from a *different distribution* and then correct the weight (e.g. easier to sample from behavior policy than target)
  - $\mathbb{E}_{\tau\sim\pi}[r(\tau)] = \int P_\pi(\tau)r(\tau)d\tau = \int P_\mu(\tau)\frac{P_\pi(\tau)}{P_\mu(\tau)}r(\tau)d\tau = \mathbb{E}_{\tau\sim\mu}[\frac{P_\pi(\tau)}{P_\mu(\tau)}r(\tau)] \approx \frac{1}{n}\sum_i \frac{P_\pi(\tau_i)}{P_\mu(\tau_i)}r(\tau_i)$
  - This can be used to perform off-policy Monte Carlo and TD Learning; use the behavior policy to generate an episode or a TD target, and then reweight the return or TD target to use for the update
    - e.g. TD:
      - $V(S_t) \leftarrow V(S_t) + \alpha(\frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}(R_{t+1} + \gamma V(S_{t+1})) - V(S_t))$

Value Function Approximation and Deep Q-Learning

- Same approaches as model-free prediction work with function approximators
  - Monte Carlo Value Function Approximation: $\Delta\mathbf{w} = \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\nabla_\mathbf{w}\hat{v}(S_t, \mathbf{w})$
    - G_t is *unbiased*, but *noisy* sample of true value
  - Temporal Difference Value Function Approximation: $\Delta\mathbf{w} = \alpha(R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}))\nabla_\mathbf{w}\hat{v}(S_t, \mathbf{w})$
    - $R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w})$ is *biased* estimate of true value since it is drawn from previous estimate
    - The gradient ascent update is an example of **semi-gradient** learning
- The model-free control algorithms are *also* similar - approximate q
  - MC: $\Delta\mathbf{w} = \alpha(G_t - \hat{q}(S_t, A_t, \mathbf{w}))\nabla_\mathbf{w}\hat{q}(S_t, A_t, \mathbf{w})$
  - Sarsa: $\Delta\mathbf{w} = \alpha(R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}))\nabla_\mathbf{w}\hat{q}(S_t, A_t, \mathbf{w})$
  - Q-Learning: $\Delta\mathbf{w} = \alpha(R_{t+1} + \gamma\max_a\hat{q}(S_{t+1}, a, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}))\nabla_\mathbf{w}\hat{q}(S_t, A_t, \mathbf{w})$
- Linear Least-Squares Prediction Algorithms:
  - LSMC: $0 = \sum_{t=1}^T \alpha(G_t - \hat{v}(S_t, \mathbf{w}))\mathbf{x}(S_t)$
    - $\mathbf{w} = (\sum_{t=1}^T \mathbf{x}(S_t)\mathbf{x}(S_t)^T)^{-1}\sum_{t=1}^T \mathbf{x}(S_t)G_t$
  - LSTD: $0 = \sum_{t=1}^T \alpha(R_{t+1} + \gamma\hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}))\mathbf{x}(S_t)$
    - $\mathbf{w} = (\sum_{t=1}^T \mathbf{x}(S_t)\mathbf{x}(S_t)^T - \gamma\mathbf{x}(S_{t+1}))^{-1}\sum_{t=1}^T \mathbf{x}(S_t)R_{t+1}$
  - LSDQ: $0 = \sum_{t=1}^T \alpha(R_{t+1} + \gamma\hat{q}(S_{t+1}, \pi(S_{t+1}), \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}))\mathbf{x}(S_t, A_t)$
    - $\mathbf{w} = (\sum_{t=1}^T \mathbf{x}(S_t, A_t)(\mathbf{x}(S_t, A_t) - \gamma\mathbf{x}(S_{t+1}, \pi(S_{t+1})))^T)^{-1}\sum_{t=1}^T \mathbf{x}(S_t, A_t)R_{t+1}$
    - Sample $S_t, A_t, R_{t+1}, S_{t+1}$ from old policy to generate an experience replay
- Deep Q-Learning: Standard Q-Learning approach, *however*, incorporates (1) **Experience replay** to address correlation between samples and (2) **Fixed Q targets** to handle non-stationary targets
  - Experience Replay: Store (s_t, a_t, r_t, s_{t+1}) suples in replay memory, and then sample an experience tuple (or batch) from this memory to compute the target value and prediction used to perform gradient descent
  - Fixed Targets: Use $\mathbf{w}^-$ as the set of weights used in the *target* and $\mathbf{w}$ as the set of weights being updated. Periodically adjust $\mathbf{w}^- \leftarrow \mathbf{w}$ (frequency is hyperparameter)
    - $\Delta\mathbf{w} = \alpha(r + \gamma\max_a\hat{Q}(s', a', \mathbf{w}^-) - \hat{Q}(s, a, \mathbf{w}))\nabla_\mathbf{w}\hat{Q}(s, a, \mathbf{w})$
  - Some improvements: **Double DQN**: use *two networks*, where one performs the action selection and another creates the target value ($\text{argmax}\hat{Q}$ term); **Dueling DQN**: branch the same network where one branch generates $V(s)$ and the other generates $A(s,a)$ so that $(Q(s,a) = A(s,a) + V(s))$ - the goal is to encourage learning which states are valuable without needing to learn the effect of action at each state; **Prioritized Experience Replay**: Prioritize experiences where there is a big difference between prediction and TD target (can use a priority queue)
- Some issues that can lead to instability in training - **function approximators**, **bootstrapping**, and **off-policy training**
- Batch-based methods are often used for training - this is more sample-efficient

Policy Optimization I: Policy Gradient and Actor-Critic

- Parameterize and optimize policy directly
  - Discrete Policy (**Softmax**): $\pi_\theta(a|s) = \frac{\exp(\phi(s,a)^T\theta)}{\sum_{a'}\exp(\phi(s,a')^T\theta)}$
    - $\nabla_\theta\log\pi_\theta(a|s) = \phi(s,a) - \mathbb{E}_{\pi_\theta}[\phi(s,.)]$
  - Continuous Policy (**Gaussian**): $\mu(s) = \phi(s)^T\theta$
    - $a \sim \mathcal{N}(\mu(s), \sigma^2)$
    - $\nabla\theta\log(\pi_\theta)(a|s) = \frac{(a-\mu(s))\phi(s)}{\sigma^2}$
- Gradient-Free Methods:
  - **Evolution Strategy**: Perturb θ with multiple noises and execute rollouts to compute $J(\theta)$ for each; set new θ to weighted sum based on scores $(\theta_{t+1} \leftarrow \theta_t + \alpha\frac{1}{n\delta}\sum_{i=1}^n J(\theta_i)\epsilon_i)$
  - **Cross-Entropy Method**: Sample θ from distribution, execute rollouts, and then update μ (distribution parameter) to maximize the likelihood of the top n% of θ generated based on score $(\mu^{(i+1)} = \text{argmax}_\mu\sum_{k\in\hat{C}}\log P_\mu(\theta^{(k)}))$
- Log Ratio Trick: $\nabla_\theta\pi_\theta(a|s) = \pi_\theta(a|s)\frac{\nabla_\theta\pi_\theta(a|s)}{\pi_\theta(a|s)} = \pi_\theta(a|s)\nabla_\theta\log\pi_\theta(a|s)$
- The objective of policy gradient is to maximize the reward from a trajectory sampled from the policy: $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^T R(s_t, a_t)] = \sum_\tau P(\tau;\theta)R(\tau)$ where $\tau = (s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T, s_T) \sim (\pi_\theta, P(s_{t+1}|s_t, a_t))$
  - To get this in a differentiable form:
    - $\nabla_\theta J(\theta) = \nabla_\theta\sum_\tau P(\tau;\theta)R(\tau) = \sum_\tau P(\tau;\theta)R(\tau)\nabla_\theta\log P(\tau;\theta)$ (**log ratio trick**)
    - $\approx \frac{1}{m}\sum_{i=1}^m R(\tau_i)\nabla_\theta\log P(\tau_i;\theta)$ (**estimate of expectation over** τ)
    - $= \frac{1}{m}\sum_{i=1}^m R(\tau_i)\nabla_\theta\log[\mu(s_0)\prod_{t=0}^{T-1}\pi_\theta(a_t|s_t)p(s_{t+1}|s_t, a_t)]$ (**decomposition of** P)
    - $= \frac{1}{m}\sum_{i=1}^m R(\tau_i)\nabla_\theta[\log\mu(s_0) + \sum_{t=0}^{T-1}\log\pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t)]$ (**can eliminate transition dynamics since they are not a function of** θ)
    - $\nabla_\theta J(\theta) \approx \frac{1}{m}\sum_{i=1}^m R(\tau_i)\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t^i|s_t^i)$
      - This is shifting the distribution (π) through parameter θ in a way to ensure future samples achieve higher scores based on the reward function - this is akin to **weighted maximum likelihood**
- **Temporal Causality**: Policy should not affect rewards from the past, so only sum *future rewards*
  - $\nabla_\theta J(\theta) = \mathbb{E}_\tau[\sum_{t=0}^{T-1}\nabla_\theta\log\pi_\theta(a_t|s_t)\sum_{t'=t}^{T-1}r_t'] \approx \frac{1}{m}\sum_{i=1}^m\sum_{t=0}^{T-1}G_t^{(i)}\cdot\nabla_\theta\log\pi_\theta(a_t^i|s_t^i)$
- **Baselines**: Reduce variance by subtracting a baseline
  - $\nabla_\theta J(\theta) = \mathbb{E}_\tau[\sum_{t=0}^{T-1}(G_t - b)\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)]$ where $b = \mathbb{E}[r_t + r_{t+1} + \ldots + r_{T-1}]$ (expected return)
- **REINFORCE**: On-policy algorithm
  - Generate episode following $\pi(\theta)$
  - For each step of episode:
    - Let G be the return from step t onwards
    - $\theta \leftarrow \theta + \alpha\gamma^t G\nabla_\theta\log\pi(A_t|S_t, \theta)$
- **Actor-Critic**: Instead of using G_t, which is sampled from Monte Carlo, use a critic to estimate the action-value function. This implies *two sets of parameters* - one for the actor (policy function), and one for the critic (value function)
  - $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{T-1}Q_w(s_t, a_t)\cdot\nabla_\theta\log\pi_\theta(a_t|s_t)]$
  - Critic parameters can be updated using any prior approach (Monte Carlo, TD, etc.)
  - Baseline can be applied here as well - the state-value function can be used to form the baseline (**advantage function**) $A^{\pi,\gamma}(s,a) = Q^{\pi,\gamma}(s,a) - V^{\pi,\gamma}(s)$
    - Rather than maintain *two sets of parameters* for just the critic, it can be recognized that the TD error is an unbiased estimate of the advantage function:
    - $\mathbb{E}_{\pi_\theta}[\delta^{\pi_\theta}|s,a] = \mathbb{E}_{\pi_\theta}[r + \gamma V^{\pi_\theta}(s')|s,a] - V^{\pi_\theta}(s) = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) = A^{\pi_\theta}(s,a)$
    - $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta\log\pi_\theta(a|s)\delta^{\pi_\theta}]$, where $\delta_v = r + \gamma V_\kappa(s') - V_\kappa(s)$
      - Just need to maintain weights for V, which can be updated using MC, TD, etc. (e.g. $\Delta\kappa = \alpha(G - V_\kappa(s))\psi(s)$)

Policy Optimization II

- Turn Policy Gradient into an *off-policy* algorithm via **importance sampling**, where the *old policy* is sampled from (using replay buffer) rather than the current one; must incorporate a constraint to limit excessive changes between subsequent policies (creates a **trust region**)
  - $J(\theta) = \mathbb{E}_{s,a\sim\pi_\theta}[r(s,a)] = \mathbb{E}_{s,a\sim\tilde{\pi}}[\frac{\pi_\theta(s,a)}{\tilde{\pi}(s,a)}r(s,a)]$ subject to $KL(\pi_{\theta_{\text{old}}}||\pi_\theta) =$

$\sum_a \pi_{\theta_{\text{old}}}(a|s) \log \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \leq \delta$

- **Natural Policy Gradient**: Use second-order optimization methods to solve aforementioned constrained objective
  - Collect trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$
  - Estimate advantages $\hat{A}_t^{\pi_k}$ using any method
  - Estimate $g = \nabla_\theta J_{\theta_i}(\theta)$ and $H = \nabla_\theta^2 KL(\theta||\theta_t)$, which can also be expressed as $F = \mathbb{E}_{\pi_\theta(s,a)}[\nabla \log \pi_\theta(s,a) \nabla \log \pi_\theta(s,a)^T]$
  - Compute update: $\theta_{k+1} = \theta_k + \sqrt{\frac{2\delta}{\hat{g}_k^T \hat{H}_k^{-1} \hat{g}_k}} \hat{H}_k^{-1} \hat{g}_k$
  - **Disadvantage**: H and H^{-1} can be very difficult to compute
- **Trust Region Policy Optimization**: Estimates $x = H^{-1}g$ by solving $Hx = g$, which is equivalent to solving $\min_x \frac{1}{2} x^T Hx - g^T x$ - do this via conjugate gradient method
  - **Disadvantage**: Computing H is still expensive, and requires a large batch of rollouts to properly approximate H
- **Proximal Policy Optimization (PPO) with Adaptive KL Penalty**: Incorporate the KL constraint into the objective directly
  - Collect set of partial trajectories $\mathcal{D}_k$ on $\pi_k = \pi(\theta_k)$
  - Estimate advantages $\hat{A}_t^{\pi_k}$ using any method
  - $\theta_{k+1} = \arg\max_\theta \mathbb{E}_{\pi_{\theta_{\text{old}}}}[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} A_t] - \beta \mathbb{E}_{\pi_{\theta_{\text{old}}}}[KL[\pi_{\theta_{\text{old}}}, \pi_\theta]]$
    - Using SGD
  - if $D_{KL}(\theta_{k+1}||\theta_k) \geq 1.5\delta$ then $\beta_{k+1} = 2\beta_K$ else $\beta_{k+1} = \beta_k/2$
    - If there are significant differences, then have a higher divergence penalty
- **PPO with Clipping**: Axe using KL divergence and instead clip the advantage function directly to avoid too large of changes in probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
  - $L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)$
- **Deep Deterministic Policy Gradient (DDPG)**: Extend DQN to *continuous* action space. Still utilize replay buffer and target network logic as with DQN
  - $a^* = \arg\max_a Q^*(s,a) \approx \arg\max_\theta Q_\phi(s, \mu_\theta(s))$
    - Train policy $\mu_\theta(s)$ to give action that maximizes $Q_\phi(s,a)$
  - **Q-Function Update**: $\min \mathbb{E}_{s,r,s',d \sim \mathcal{D}}[(r + \gamma Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s')) - Q_\phi(s,a))^2]$
  - **Policy Update**: $\max_\theta \mathbb{E}_{s \sim \mathcal{D}}[Q_\phi(s, \mu_\theta(s))]$
  - **Disadvantage**: Tends to *dramatically overestimate Q-values*
- **Twin Delayed DDPG (TD3)**: Addresses issues with DDPG
  - **Double-Q**: Learn two Q functions, Q_{ϕ_1} and Q_{ϕ_2} using the same aforementioned loss function, but have the target be the smaller of the two: $y(r,s') = r + \gamma \min_{i=1,2} Q_{\phi_{i,targ}}(s', a_{TD3}(s'))$
    - Note that there are still *two target networks* for each Q function
  - **Target Policy Smoothing**: $a_{TD3}(s') = \text{clip}(\mu_{\theta,\text{targ}}(s') + \text{clip}(\epsilon, -c, c), a_{low}, a_{high}), \epsilon \sim \mathcal{N}(0, \sigma)$
    - This regularizes by avoiding incorrect sharp peak for some actions output by the policy
- **Soft Actor-Critic (SAC)**: Incorporates **entropy regularization** to ensure there is a tradeoff between expected return and entropy in the policy - this encourages exploration
  - Q-Function Update:
    - $L(\phi_i, \mathcal{D}) = \mathbb{E}[(Q_\phi(s,a) - y(r,s',d))^2]$
    - $y(r,s') = r + \gamma(\min_{j=1,2} Q_{\phi_{targ,j}}(s', \hat{a}') - \alpha \log \pi_\theta(\hat{a}'|s'))$
    - $\hat{a}' \sim \pi_\theta(.|s')$
      - No need to reparameterize here because the gradient is for ϕ, not θ
  - Policy Update:
    - $\max_\theta E_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}}[\min_{j=1,2} Q_{\phi_j}(s, \hat{a}_\theta(s,\epsilon)) - \alpha \log \pi_\theta(\hat{a}_\theta(s,\epsilon)|s)]$
    - To avoid the dependency of sampling from the policy in the objective, we used a **reparameterization trick** to instead sample over noise
      - $\hat{a}_\theta(s,\epsilon) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon), \epsilon \sim \mathcal{N}(0, I)$

Model-Based Reinforcement Learning

- Learn a **model** that can be used to *plan* a better policy - this is much more *sample efficient*, but can be difficult to learn since now there are two sources of error
  - Model learns to predict S_{t+1} and R_{t+1} in a supervised fashion from collected trajectories
  - Planning: Sample experiences from the model and then use model-free reinforcement learning algorithms (Monte Carlo, Sarsa, Q-Learning) to the sampled experiences to learn
- **Dyna-Q** (Model-Based Value Optimization):
  - Execute action A based on policy (e.g. ϵ-greedy), and observe resultant R and S'
  - $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
  - Update model from collected R and S' (e.g. regression)
  - Repeat n times:
    - $S \leftarrow$ random previously observed state; $A \leftarrow$ random action previously taken in S
    - $R, S' \leftarrow \text{Model}(S,A)$
    - $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$
- **Model-Based Policy Optimization**:
  - **Linear-Quadratic Regulator** methods can be used to solve $\min_{a_1,\ldots,a_t} \sum_{t=1}^T c(s_t, a_t)$ subject to $s_t = f(s_{t-1}, a_{t-1})$, where c is the cost function (can use *negative reward*) and s_t is the transition dynamics
  - Algorithm (Planning Incorporated):
    - Run base policy $\pi_0(a_t|s_t)$ to collect $\mathcal{D} = \{(s,a,s')_i\}$
    - Loop every step:
      - Loop every N steps:
        - Learn dynamics model $s' = p(s,a)$ to minimize $\sum_i ||f(s_i, a_i) - s'_i||^2$
        - Plan through $f(s,a)$ to choose actions (use LQR method)
        - Execute first planned action and observe resulting state
        - Append (s,a,s') to dataset $\mathcal{D}$
  - Algorithm (Learning and Planning Incorporated):
    - Run base policy $\pi_0(a_t|s_t)$ to collect $\mathcal{D} = \{(s,a,s')_i\}$
    - Loop:
      - Learn dynamics model $f(s,a)$ to minimize $\sum_i ||f(s_i, a_i) - s'_i||^2$
      - Backpropagate through $f(s,a)$ into policy to optimize $\pi_\theta(a_t|s_t)$
      - Run $\pi_\theta(a_t|s_t)$, appending visited (s,a,s') to $\mathcal{D}$

Imitation Learning

- **Imitation Learning** becomes a *supervised learning* of the policy network, but standard behavioral cloning is limited since the policy can run into off-course situations
- **Dataset Aggregation (DAgger)**: Initially collect human data and train π_θ from human data. Then, run π_θ to get a new dataset that is then *labeled by humans with correct actions*. Finally, aggregate the newly labeled dataset with the original (already labeled dataset)
  - Human annotation is expensive - could try to substitute this out with an offline brute-force search (e.g. other, much slower algorithm)
- **HG-DAgger**: Same approach as dagger, but instead *gate* human intervention - if human decides to take control, then record their state and actions into the dataset
  - One drawback here is that the bad agent actions are ignored in the original formulation: $\mathcal{L}_{\text{HG-DAgger}(\pi)} = \mathbb{E}_{(s,a_h) \sim \mathcal{D}_{\text{takeover}}}[||\pi(s) - a_h||^2]$
- **Proxy Value Propagation**: In cases of intervention, discourage agent action a_n while promoting human action a_h
  - $J^{PV}(\theta) = \mathbb{E}_{(s,a_n,a_h)}[|Q_\theta(s,a_h) - 1|^2 + |Q_\theta(s,a_n) + 1|^2]$
  - $J^{TD}(\theta) = \mathbb{E}_{(s,a,s')}[|Q_\theta(s,a) - \gamma \max_{a'} Q_\theta(s',a)|^2]$
  - $J(\theta) = J^{PV}(\theta) + J^{TD}(\theta)$ (Learn policy via Q optimized from J)
- **Predictive Preference Learning**: Provide a visualization of agent trajectory, and leverage preference learning on human trajectories when they decide to takeover
  - $L(\pi) = \mathbb{E}_{s,a^+,a^-}[\log \sigma(\beta \log \frac{\pi(a^+|s)}{\pi(a^-|s)})]$
- **Adaptive Intervention Mechanism (AIM)**: Learn an uncertainty estimator such that the agent requests help when necessary (e.g. when $Q_\theta(s,a_n) < \delta$ quantile of $Q_\theta(s,a_n)$)
- **Inverse Reinforcement Learning**: Provide expert data to learn a cost/reward function to explain expert behaviors
  - **Maximum Entropy IRL**: Observed trajectories: $\mathcal{D} = \{\tau_i\}$, model reward linearly $R(\tau) = \sum_t w^T \phi(s_t, a_t)$
    - Optimize for distribution that matches expert behavior but is otherwise maximally uncertain: $\max_{p(\tau)} H(p) = -\sum_\tau p(\tau) \log p(\tau)$ subject to $\mathbb{E}_p[\phi(\tau)] = \hat{\phi}_{expert}$
    - Result: $p(\tau|w) = \frac{1}{Z(w)} \exp(R(\tau))$ where $Z(w) = \sum_\tau \exp(R(\tau))$ (expert trajectories exponentially more likely if rewards are high)
    - $\mathcal{L}(w) = \sum_{\tau \in \mathcal{D}}[R(\tau) - \log Z(w)]$
  - **Generative Adversarial Imitation Learning (GAIL)**: Train a discriminator to predict whether a given state s and action s is sampled from the demonstrations M or generated by running the policy π
    - $\arg\min_D -\mathbb{E}_{d^M(s,a)}[\log(D(s,a))] - \mathbb{E}_{d^\pi(s,a)}[\log(1 - D(s,a))]$
    - Train policy with $r_t = -\log(1 - D(s_t, a_t))$
    - If there are state-only demonstrations, then $\arg\min_D -\mathbb{E}_{d^M(s,s')}[\log(D(s,s'))] - \mathbb{E}_{d^\pi(s,s')}[\log(1 - D(s,s'))]$ and $r(s_t, s_{t+1}) = \max[0, 1 - 0.25(D(s_t, s_{t+1}) - 1)^2]$
- Common paradigm is to **use imitation learning for pretraining** and then **finetune with standard methods (e.g. policy gradient)**
  - To avoid forgetting demonstrations, try to leverage off-policy reinforcement learning so that the demonstrations can be injected into the off-policy samples (maybe with *importance sampling*)

System Design

- **Model Parallelism**: Different machines are responsible for computation in parts of a *single network*
- **Data Parallelism**: Different machines have the complete copy of the model, but get different portions of the data
- Approaches to Parameter Updating in Distributed Systems:
  - **Parameter Averaging**: Distribute copy of parameters to each worker, then train each worker on subset of data, and set up the global parameters to average the parameters from each worker - repeat
  - **Distributed Stochastic Gradient Descent**: Transfer just updates (gradient) to the parameter server, and server periodically transfers weights to worker machines
  - **Synchronous Updates** have agents training in parallel but then wait until all updates are collected before enforcing on global network; **Asynchronous Updates** have the global network updated periodically by individual workers but not necessarily simulatenously
  - **Hogwild**: Normally, lock-based SGD requires a lock on current state of parameters before updating; however, since updates are often sparse there is no need for the locks since there is still convergence
- **Distributed Architectures**:
  - Separate out replay memory, actors, parameter servers, and learners

Assignment 1

```python
# Gym Environment Semantics
env = gym.make(env_name)
obs, info = env.reset(seed=seed)
action = policy(obs)
observation, reward, terminated, truncated, info = env.step(action)
done = terminated or truncated
transitions = self.env.unwrapped.P[state][act]
for prob, next_state, reward, done in transitions:
  ...


# Policy Iteration: Note that train() is updating value and then policy
random_mapping = np.random.randint(0, self.action_dim, size=(self.obs_dim))
self.policy: Callable = lambda obs: random_mapping[obs]
# Update Value Function
while True:
  old_table = self.value_table.copy()
  for state in range(self.obs_dim):
    action = self.policy(state)
    state_value = 0
    for transition in self._get_transitions(state, action):
      # Unpack transitions (omitted here)
      if done:
        state_value += prob * reward
      else:
        state_value += prob * (reward + self.gamma *
old_table[next_state])
    self.value_table[state] = state_value
  if np.abs(old_table - self.value_table).sum() < self.eps:
    break
# Update Policy
policy_table = np.zeroes([self.obs_dim, ], dtype=int)
for state in range(self.obs_dim):
  state_action_values = [0] * self.action_dim
  for action in range(self.action_dim):
    value = 0
    for transition in self._get_transitions(state, action):
      # Omit unpacking logic here
      if done:
        value += prob * reward
      else:
        value += prob * (reward + self.gamma *
self.value_table[next_state])
    state_action_values[action] = value
  policy_table[state] = np.argmax(state_value)
self.policy = lambda obs: policy_table[obs]


# Value Iteration: Note that training is just updating value function
# Update Value Function
old_table = self.value_table.copy()
for state in range(self.obs_dim):
  state_value = 0
  state_action_values = [0] * self.action_dim
  for action in range(self.action_dim):
    value = 0
    for transition in self._get_transitions(state, action):
      # Omit unpacking logic here
      if done:
        value += prob * reward
      else:
        value += prob * (reward + self.gamma * old_table[next_state])
    state_action_values[action] = value
  state_action_valuesp[action] = value
  self.value_table[state] = max(state_action_values)
```

Assignment 2

```python
# Q-Learning
# Epsilon Greedy Policy
if np.random.uniform(0, 1) < self.eps:
  action = self.env.action_space.sample()
else:
  action = np.argmax(self.table[obs])
return action
# Training
for t in range(self.max_episode_length):
  act = self.compute_action(obs) # Epsilon-Greedy
  next_obs, reward, terminated, truncated, info = self.env.step(act)
  done = terminated or truncated
  td_error = reward + self.gamma * np.max(self.table[next_obs]) -
self.table[obs][act]
  new_value = self.table[obs][act] + self.learning_rate * td_error
  self.table[obs][act] = new_value
  obs = next_obs
  if done:
    break


# DQN
# ExperienceReplayMemory is a dequeue with push and sample methods
# Pytorch Model Syntax
self.action_value = nn.Sequential(
  nn.Linear(num_inputs, hidden_units),
  nn.ReLU(),
  nn.Linear(hidden_units, hidden_units),
  nn.ReLU(),
  nn.Linear(hidden_units, num_outputs)
)
# DQN Setup
# Set both network and target networks to eval mode (omitted here)
self.network = PytorchModel(self.obs_dim, self.act_dim, self.hidden_dim)
self.target_network = PytorchModel(self.obs_dim, self.act_dim)
self.target_network.load_state_dict(self.network.state_dict())
self.loss = nn.MSELoss()
# Training Function Snippets
act = self.compute_action(processed_obs) # Epsilon-Greedy
for t in range(self.max_episode_length):
  next_obs, reward, truncated, terminated, truncated, info =
self.env.step(act)
  done = terminated or truncated
  self.memory.push((processed_obs, act, reward, next_processed_obs, done))
  act = self.compute_action(next_processed_obs)
  batch = self.memory.sample(self.batch_size)
  # Convert batches to tensors using np.stack and to_tensor - omitted
  with torch.no_grad():
    Q_t_plus_one = self.target_network(next_state_batch).max(dim=1)[0]
    Q_objective = reward_batch.squeeze(0) + (1 - done_batch.squeeze(0)) *
self.gamma * Q_t_plus_one

  self.network.train()
  Q_t = self.network(state_batch).gather(1,
action_batch.long().squeeze(0).unsqueeze(1)).squeeze(1)
  self.optimizer.zero_grad()
  loss = self.loss(Q_t, Q_objective)
  loss.backward()
  self.optimizer.step()
  self.network.eval()

  if (update_weights): # omit log
    self.target_network.load_state_dict(self.network.state_dict())
    self.target_network.eval()


# Policy Gradient Network Snippets
def forward(self, obs):
  logit = self.network(obs)
  dist = torch.distributions.Categorical(logits=logit)
  action = dist.sample()
  return action
def log_prob(self, obs, act):
  logit = self.network(obs)
  dist = torch.distributions.Categorical(logits=logit)
  log_prob = dist.log_prob(act)
  return log_prob
# Update
self.network.train()
self.optimizer.zero_grad()
log_probs = self.compute_log_probs(flat_obs, flat_act)
loss = -torch.mean(log_probs * advantages)
loss.backward()
torch.nn.utils.clip_grad_norm_(self.network.parameters(),
self.config["clip_gradient"])
self.optimizer.step()
self.network.eval()
# Sample collection logic
while iter_timesteps <= self.config["train_batch_size"]
  ...
  while True:
    act = self.compute_actions(obs)
    next_obs, reward, terminated, truncated, step_info = self.env.step(act)
    # Append everything
# Sample Processing
for reward_list in samples["reward"]
  ...
  for t in reversed(range(len(reward_list))):
    Q = reward_list[t] + self.config["gamma"] * Q
    returns[t] = Q
  values.append(returns) # technically values are advantages here
```

Assignment 3

```python
# TD3 Actor (Continuous)
def forward(self, state):
  a = F.relu(self.l1(state))
  a = F.relu(self.l2(a))
  return self.max_action * torch.tanh(self.l3(a))
# TD3 Critic
def forward(self, state, action):
    sa = torch.cat([state, action], 1)
    q1 = F.relu(self.l1(sa))
    q1 = F.relu(self.l2(q1))
    q1 = self.l3(q1)
    q2 = F.relu(self.l4(sa))
    q2 = F.relu(self.l5(q2))
    q2 = self.l6(q2)
    return q1, q2
# TD3 Trainer Snippets (Actor, Actr Target, Critic, and Critic Targets)
state, action, next_state, reward, not_done =
replay_buffer.sample(batch_size)
with torch.no_grad():
    noise = (torch.normal(0, self.policy_noise,
size=action.shape).to(device)).clamp(-self.noise_clip, self.noise_clip)
  next_action = (self.actor_target(next_state) +
noise).clamp(-self.max_action, self.max_action)
  target_Q1, target_Q2 = self.critic_target(next_state, next_action)
  target_Q = torch.min(target_Q1, target_Q2)
  target_Q = reward + not_done * self.gamma * target_Q
current_Q1, current_Q2 = self.critic(state, action)
critic_loss = F.mse_loss(current_Q1, target_Q) + F.mse_loss(current_Q2,
target_Q)
self.critic_optimizer.zero_grad()
critic_loss.backward()
self.critic_optimizer.step()
if (update_actor):
  actor_loss = -self.critic.Q1(state, self.actor(state)).mean()
  self.actor_optimizer.zero_grad()
  actor_loss.backward()
  self.actor_optimizer.step()
  # Update target models here (rolling update)
  # e.g. target_param.data.copy_(self.tau * param.data + (1 - self.tau) *
  target_param.data)


# PPO
# Rollout Storage GAE Advantage Computing
self.value_preds[-1] = next_value
gae = 0
for step in reversed(range(self.rewards.size(0))):
  delta = self.rewards[step] + gamma * self.value_preds[step + 1] *
self.masks[step + 1] - self.value_preds[step]
  gae = delta + gamma * self.gae_lambda * self.masks[step + 1] * gae
  self.returns[step] = gae + self.value_preds[step]
# PPO Trainer Snippets
def compute_action(self, obs, deterministic=False):
  actions, action_log_probs = None, None
  if self.discrete:
    logits, values = self.model(obs)
    dist = Categorical(logits = logits)
    actions = dist.probsl.argmax(dim=-1) if determinstic else dist.sample()
    action_log_probs = dist.log_prob(actions)
  else:
    means, log_std, values = self.model(obs)
    std = log_std.exp()
    dist = torch.distributions.Normal(means, std)
    actions = means if deterministic else dist.sample()
    action_log_probs = dist.log_prob(actions).sum(dim=-1)
  return values, actions, action_log_probs
def compute_loss(self, sample):
  ...
  # Function gets logits, values from model, creates distribution, then uses
  dist.entropy() to get entropy
  values, action_log_probs, dist_entropy =
self.evaluate_actions(observations_batch, actions_batch)
  ...
  ratio = torch.exp(action_log_probs - old_action_log_prob) # pi / pi_old
  policy_loss = -torch.min(ratio * adv_targ, torch.clamp(ratio, 1.0 -
self.clip_param, 1.0 + self.clip_param) * adv_targ).mean()
  value_loss = F.mse_loss(values, return_batch, reduction='none').mean()
  loss = policy_loss + self.config.value_loss_weight * value_loss -
self.config.entropy_loss_weight * dist_entropy
  loss = loss.mean()
  return loss, policy_loss_mean, value_loss_mean, torch.mean(dist_entropy),
  torch.mean(ratio)
```

Assignment 4

```python
# Behavior Cloning Loss
pi_s = policy(states)
loss = F.mse_loss(pi_s, actions, reduction='mean')
return loss
# HG Dagger Training
for step in range(HG_STEPS):
    idx = np.random.choice(len(recov_data), BATCH_SIZE)
    hg_loss = compute_bc_loss(hg_model, recov_data.states[idx].to(device,
recov_data.actions[idx].to(device)))
# DPO Loss
pi = policy(states)
with torch.no_grad():
  pi_ref = ref_policy(states)
pi_neg_error = ((pi - neg_actions)**2).sum(dim=-1)
pi_pos_error = ((pi - pos_actions)**2).sum(dim=-1)
ref_neg_error = ((pi_ref - neg_actions)**2).sum(dim=-1)
ref_pos_error = ((pi_ref - pos_actions)**2).sum(dim=-1)
loss = -F.logsigmoid((pi_neg_error - pi_pos_error) - (ref_neg_error -
ref_pos_error)).mean()
```